SIGNHIFY STUDIO | PORTFOLIO + REVENUE BUILDS

High-Yield Free-Stack Projects

10 ranked SaaS, web-app and mobile builds - each with a complete, copy-paste, production-ready building prompt. Built only on free and open-source technology.

Prepared for: Piyush Raj Singh
Founder & Technical Lead, Signhify Studio
Generative AI Engineer | Full-Stack Developer

Stack focus: Python - TypeScript - Next.js - FastAPI - LangChain / RAG - Supabase / pgvector - Three.js / WebGL - Expo

Read this first:
No project in this document guarantees income - let alone unlimited money. What these builds do is stack the odds: they prove the exact skills $100K+ remote roles pay for, and they each sell a clear outcome a business will pay a setup fee for. Job first. Services second. Templates third. Subscriptions last.

7 November 2026 | Zero-capital edition: every stack, limit and free alternative is named. No paid service is required to ship any project here.

START HERE

Money reality: the ladder, in order

> **No guarantees.** Revenue figures in this document are **targets based on common market pricing for comparable services**, not promises. Actual income depends on finishing, distribution, sales conversations and timing. Anyone who promises unlimited money from a project list is selling you the list.

1. High-pay job first - fastest, most reliable return

A finished, deployed, well-documented project is interview evidence. One strong flagship beats ten half-built demos. This is why Project #1 - the build you have already started - ranks first, and why build order matters more than idea count.

2. Paid setup services second - cash before subscriptions

Businesses pay faster for a done-for-you outcome than for a login. A WhatsApp agent installed for a clinic, or a support copilot wired to a store's help docs, can be sold as a fixed setup fee (targets: $250-$2,500 per install, depending on buyer and scope) plus optional monthly care. You deliver with the same code you show in your portfolio.

3. Templates third - package what already works

Once a build has run for a real client, strip the client data, document it, and sell the starter on Gumroad or Lemon Squeezy (targets: $29-$149 per copy). Template income is slow at first and costs almost nothing to keep alive.

4. Subscriptions last - only after proof

SaaS subscriptions (targets: $19-$99 / month per customer) need churn management, support and steady acquisition. Pursue them after a setup client asks, "can you just run this for us?" - not before.

> **How to use this PDF:** (1) Finish Project #1 before starting anything else - it is already at Day 4 of 15. (2) Copy the prompt for the project you are building into your coding assistant exactly as written; do not merge prompts. (3) Ship the MVP list only. Every "later" feature is a delay. (4) Record a 2-minute demo the day it deploys. (5) Only then open the next project.

Scoring used in the shortlist: Job Signal (what a hiring manager infers), Revenue (nearest path to a paid outcome), Speed (days to a demoable MVP), and $0 Fit (can it run entirely inside free tiers or self-hosted open source).

TOOLBOX

The $0 stack: what to use, where it ends, what replaces it

Layer	Use this (free)	Where the free tier ends	Free alternative
Hosting - web	Cloudflare Pages + Workers (free tier) / Vercel Hobby	Pages/Workers free tier has request and CPU limits; Vercel Hobby is for non-commercial / personal use only - do not host paid client work on Hobby	Cloudflare Pages for commercial work; Oracle Cloud Always Free ARM VM (Ampere) with Docker + Caddy for anything long-running
Hosting - backend	Oracle Cloud Always Free ARM VM	VM quotas and idle policies change; keep infra as code so you can move in an hour	Self-host on the Oracle VM: Docker Compose, Caddy, GitHub Actions deploy
Database	Supabase free project (Postgres + pgvector + Auth + Storage)	Free projects pause after inactivity and have database / storage caps; two free projects per org at time of writing	Self-hosted Postgres 16 + pgvector on the Oracle VM (Docker), with nightly pg_dump to Cloudflare R2
LLM inference	Groq free tier, Cerebras free tier, Google Gemini Flash free tier (AI Studio), OpenRouter free models, Ollama (local)	Free tiers carry daily / per-minute rate caps and models rotate; never hard-code one provider	Multi-provider gateway (LiteLLM, open source) with fallbacks; Ollama + a small open model locally for dev and offline demos
Embeddings	FastEmbed / BGE-small via Ollama, Gemini embedding free tier	Local embedding is slower on CPU; batch ingestion overnight if needed	Self-hosted embedding service on the Oracle VM; cache embeddings in pgvector
Voice / video	Kokoro TTS (open weights), Whisper / faster-whisper (local), FFmpeg	Kokoro and Whisper run fine on CPU for short clips; long videos queue	Render on the Oracle VM or locally; never depend on a paid TTS / transcription API
Auth / storage / analytics	Supabase Auth, Cloudflare R2 (free tier, 10 GB class), Umami (self-hosted), Uptime Kuma (self-hosted)	R2 free tier has storage and operations caps; monitor usage monthly	Self-host Umami + Uptime Kuma on the Oracle VM; R2 lifecycle rules to delete temp renders
Automation / CI	GitHub Actions (free minutes for public repos), Docker, Playwright	Private-repo free minutes are limited; keep product repos public where IP allows	Self-hosted runner on the Oracle VM if minutes run out

Licensing traps - read before you build on these:

- **Remotion** is source-available, not free for every commercial use - companies above its free threshold need a licence. Prefer FFmpeg + your own rendering code for commercial video products.
- **n8n** is fair-code: fine for your own internal use, restricted for reselling / embedding as a hosted service for others. Build client automations in code (FastAPI / workers) if you will charge for them.
- **XTTS, F5-TTS and FLUX-dev** carry non-commercial terms on their weights / licences - do not use them in a paid product. Kokoro TTS, Whisper and SDXL / FLUX-schnell (check the current licence per model) are the safer free routes listed per project.
- **Vercel Hobby** is non-commercial only. Demos and portfolio: fine. A paying client's production app: use Cloudflare Pages or your Oracle VM.

Free-tier quotas change. Before any client launch, re-check the provider's current limits page and write the limit into that project's README. Every prompt in this document tells the builder to add a usage guardrail so a free tier is never silently exceeded.

SHORTLIST

10 projects, ranked: job signal first

Scores are out of 10 and reflect your position today: an existing flagship at Day 4, a Signhify client pipeline, and a job sprint already running. A project that proves senior-level judgement (evaluation, approvals, guardrails) outranks a project that only proves you can call an API.

#	Project	Job signal	Revenue	Speed	$0 fit	Build estimate
1	**Signhify GTM Autopilot OS**	10	9	7	10	11 days left (Days 5-15)
2	**RAG Support Copilot + Eval Dashboard**	10	8	8	10	7-10 days
3	**WhatsApp AI Sales & Support Agent**	8	10	9	8	5-7 days
4	**3D Product Configurator / Showroom**	10	9	6	9	10-14 days
5	**AI Lead Recovery & Outreach Agent**	8	9	8	9	6-8 days
6	**JobAutopilot Pro**	9	6	8	10	7-9 days
7	**Faceless Shorts / Reels Factory**	7	6	7	9	6-9 days
8	**AI Meeting-to-CRM Copilot**	9	8	8	10	5-7 days
9	**Freelancer Client OS**	7	7	9	10	5-7 days
10	**AI Interview Prep Coach (PWA)**	8	6	7	10	7-10 days

> **The order is the strategy:** #1 is already started - finishing it is the highest-value move in this document. #2 proves the RAG + evaluation depth that AI roles interview for. #3 or #5 is the first services-led build, because a local business or agency can pay a setup fee in week one. Everything after that is sequencing, not urgency.

What "high-yield" means here

- **For jobs:** deployed URL, public repo with tests and CI, an eval or metrics dashboard, and a 2-minute demo. That package is what moves an application out of the tutorial pile.
- **For revenue:** a named buyer with a painful, recurring problem and a setup-fee offer. If you cannot name the buyer in one sentence, the project is a demo, not a business.
- **For cost:** $0 to build and $0 to run at demo scale. Every project names where its free tier ends.

PROJECT 1 OF 10

1 Signhify GTM Autopilot OS

Content research to draft to approval to publish - an autopilot for agencies that never posts without a human yes.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Agencies and founders know content drives leads, but research, drafting, checking and scheduling eat 8-12 hours a week. Unreviewed AI content is the risk: one invented claim damages the brand.

Target customer

Marketing agencies, founder-led studios and SaaS teams (10-50 people) who publish weekly and need an approval gate.

> **Why it pays (targets, not guarantees):** Setup service target: $750-$2,500 to install and brand it for an agency. White-label licence target: $149-$499 one-time. Care plan target: $99-$299/month. Your own Signhify pipeline is the live proof.

Exact free tech stack

Next.js 15 + TypeScript (existing), Supabase Postgres (12 tables, RLS, approval gate already built), LiteLLM gateway over Groq / Gemini Flash free tiers + Ollama fallback, Cloudflare Pages or Oracle VM hosting, GitHub Actions CI. Free-tier end: Supabase pauses when inactive, LLM daily caps - gateway queues and retries across providers.

MVP feature list - build only this

- Topic engine: source scan, scoring, dedupe (built)
- Draft generator with brand voice profile and source citations
- Fact-check gate: approval score, zero contradicted claims allowed to pass (built)
- Human approval queue - nothing publishes without approval
- Scheduler + publishers for LinkedIn / X / blog via official APIs the user connects
- Analytics view: posts, approvals, edits, time saved

Portfolio signal

A hiring manager sees a real multi-step agentic system with an evaluation gate, RLS, CI and a live deployment - not a chatbot wrapper. The approval gate is the senior detail: it shows you design for trust, not just generation.

Build estimate: 11 days remaining (Days 5-15 of the existing plan) | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 1 - COPY-PASTE BUILDING PROMPT

Signhify GTM Autopilot OS - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: Signhify GTM Autopilot OS - Content research to draft to approval to publish - an
autopilot for agencies that never posts without a human yes.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js 15 + TypeScript (existing), Supabase Postgres (12 tables,
RLS, approval gate already built), LiteLLM gateway over Groq / Gemini Flash free tiers + Ollama
fallback, Cloudflare Pages or Oracle VM hosting, GitHub Actions CI. Free-tier end: Supabase
pauses when inactive, LLM daily caps - gateway queues and retries across providers.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): workspaces, users,
brand_profiles, sources, topics, drafts, fact_checks, approvals, schedules, publications,
analytics_events, audit_log

API ROUTES (implement all, with input validation and structured errors): /api/intake,
/api/topics/generate, /api/drafts, /api/drafts/[id]/factcheck, /api/approvals, /api/schedule,
/api/publish, /api/analytics

MVP FEATURES (build ONLY these for v1): Topic engine: source scan, scoring, dedupe (built);
Draft generator with brand voice profile and source citations; Fact-check gate: approval score,
zero contradicted claims allowed to pass (built); Human approval queue - nothing publishes
without approval; Scheduler + publishers for LinkedIn / X / blog via official APIs the user
connects; Analytics view: posts, approvals, edits, time saved.

SPECIFIC RULES FOR THIS PROJECT: CONTINUE the existing repository - do not restart it. Audit
Days 1-4 code first, list gaps against this spec, then implement Days 5-15 in order: LLM
adapters, approval UI, scheduler, publishers, analytics, hardening. Preserve the fact-check
rule: approval score >=70 and zero contradicted claims.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 2 OF 10

2 RAG Support Copilot with Eval Dashboard

A support copilot for SaaS and Shopify stores that answers only from the store's own docs - and proves it with eval scores.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Small SaaS and e-commerce teams answer the same 40 questions daily, while generic chatbots invent refund and shipping answers. Owners fear wrong answers more than slow answers.

Target customer

Shopify / WooCommerce stores and micro-SaaS teams with a help centre, FAQ or PDF manuals and no support hire.

> **Why it pays (targets, not guarantees):** Setup target: $500-$1,500 per store (ingest docs, brand the widget, tune thresholds). Monthly care target: $49-$199. Template target: $79-$149 for the self-host starter.

Exact free tech stack

Next.js + TypeScript widget, FastAPI ingestion service, Supabase Postgres + pgvector, FastEmbed (BGE-small) locally, Groq / Gemini Flash free tier via LiteLLM, Cloudflare Pages + Oracle VM. Free-tier end: embedding and LLM rate caps - cache answers for repeated questions.

MVP feature list - build only this

- Doc ingestion: URLs, PDFs, CSV FAQs into chunked pgvector index
- Grounded chat widget: answer + cited sources, 'I don't know' below confidence threshold
- Eval dashboard: golden question set, groundedness, citation accuracy, refusal accuracy
- Handoff: unanswered questions become a ticket / email draft
- Admin: re-index, threshold tuning, conversation log

Portfolio signal

Evaluation is the differentiator in AI interviews. Groundedness scores, a golden dataset and refusal behaviour show production RAG judgement that most applicants cannot demonstrate.

Build estimate: 7-10 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 2 - COPY-PASTE BUILDING PROMPT

RAG Support Copilot with Eval Dashboard - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: RAG Support Copilot with Eval Dashboard - A support copilot for SaaS and Shopify
stores that answers only from the store's own docs - and proves it with eval scores.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js + TypeScript widget, FastAPI ingestion service, Supabase
Postgres + pgvector, FastEmbed (BGE-small) locally, Groq / Gemini Flash free tier via LiteLLM,
Cloudflare Pages + Oracle VM. Free-tier end: embedding and LLM rate caps - cache answers for
repeated questions.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): tenants, documents,
chunks(vector), conversations, messages, eval_sets, eval_questions, eval_runs, tickets

API ROUTES (implement all, with input validation and structured errors): /api/ingest,
/api/chat, /api/chat/feedback, /api/evals/run, /api/evals/results, /api/tickets,
/api/admin/reindex

MVP FEATURES (build ONLY these for v1): Doc ingestion: URLs, PDFs, CSV FAQs into chunked
pgvector index; Grounded chat widget: answer + cited sources, 'I don't know' below confidence
threshold; Eval dashboard: golden question set, groundedness, citation accuracy, refusal
accuracy; Handoff: unanswered questions become a ticket / email draft; Admin: re-index,
threshold tuning, conversation log.

SPECIFIC RULES FOR THIS PROJECT: The eval dashboard is not optional - it is the product. Ship
at least 50 golden questions per demo tenant, run evals on every prompt or model change, and
block deployment in CI if groundedness drops below the stored baseline.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 3 OF 10

3 WhatsApp AI Sales & Support Agent

A WhatsApp agent for local businesses that answers, qualifies and books - handed over as a paid install, not a subscription pitch.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Clinics, real-estate agents and coaching centres lose enquiries that arrive after hours on WhatsApp. Staff answer price and availability questions all day instead of closing.

Target customer

Clinics, property agents, coaching / tuition centres and salons in India and the Gulf - businesses whose customers already live on WhatsApp.

> **Why it pays (targets, not guarantees):** Setup-fee model first. Install target: $250-$750 per business (catalogue, FAQs, booking flow, staff handoff). Add-on target: $99-$299 for a second language or branch. Care target: $29-$79/month. Targets, not guarantees - close one business before building extras.

Exact free tech stack

FastAPI + Python, WhatsApp Cloud API (Meta, free tier for service conversations within limits - verify current pricing window before launch), Supabase Postgres + pgvector for FAQs, Groq free tier, Oracle VM hosting, Next.js admin panel. Free-tier end: WhatsApp conversation pricing applies beyond free service windows - the client connects their own Meta account and bears any Meta charges; state this in writing before install.

MVP feature list - build only this

- FAQ answers grounded in the business's own price list and policies
- Lead qualification: service, budget, preferred time, location
- Booking intent capture + staff notification summary
- Human handoff keyword and office-hours mode
- Admin panel: FAQs, leads table, conversation log

Portfolio signal

Shows applied GenAI in a channel real businesses use, plus integration discipline: webhooks, idempotency, handoff design and client-owned accounts.

Build estimate: 5-7 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 3 - COPY-PASTE BUILDING PROMPT

WhatsApp AI Sales & Support Agent - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: WhatsApp AI Sales & Support Agent - A WhatsApp agent for local businesses that
answers, qualifies and books - handed over as a paid install, not a subscription pitch.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): FastAPI + Python, WhatsApp Cloud API (Meta, free tier for service
conversations within limits - verify current pricing window before launch), Supabase Postgres +
pgvector for FAQs, Groq free tier, Oracle VM hosting, Next.js admin panel. Free-tier end:
WhatsApp conversation pricing applies beyond free service windows - the client connects their
own Meta account and bears any Meta charges; state this in writing before install.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): businesses, faqs,
chunks(vector), leads, conversations, messages, appointments, handoffs

API ROUTES (implement all, with input validation and structured errors): /webhooks/whatsapp
(GET verify, POST events), /api/leads, /api/faqs, /api/appointments, /api/handoff,
/api/admin/conversations

MVP FEATURES (build ONLY these for v1): FAQ answers grounded in the business's own price list
and policies; Lead qualification: service, budget, preferred time, location; Booking intent
capture + staff notification summary; Human handoff keyword and office-hours mode; Admin panel:
FAQs, leads table, conversation log.

SPECIFIC RULES FOR THIS PROJECT: The business must use its OWN WhatsApp Business / Meta account
and phone number. Never register client traffic under your personal account. Add duplicate-
message protection (message ID idempotency) and a hard rule: medical, legal or payment
questions hand off to a human.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 4 OF 10

4 3D Product Configurator / Virtual Showroom

A 60 FPS WebGL configurator, white-labelled for e-commerce and property - buyers change finishes and see the product, live.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Furniture, interiors and property sellers lose buyers to static photos. A configurator lifts engagement, but agencies quote high-ticket custom builds most sellers cannot test cheaply.

Target customer

Furniture / interiors brands, modular-kitchen firms, real-estate developers and the agencies serving them. This is your Signhify differentiator - lead with frame rate and load time.

> **Why it pays (targets, not guarantees):** White-label build target: $1,500-$5,000 per brand / project (one configurator, their products). Agency overflow target: fixed quote per scene. Template target: $99-$249 for a starter scene kit. High-ticket B2B - one close funds months of runway.

Exact free tech stack

Next.js + TypeScript, Three.js / React Three Fiber, glTF models (Blender, free), Draco compression, Supabase for product / lead data, Cloudflare Pages + R2 for models. Free-tier end: R2 free storage / operations caps - compress models, lazy-load scenes, set a per-scene size budget (target under 8 MB initial load).

MVP feature list - build only this

- One hero product scene at 60 FPS on a mid-range phone (show the FPS meter)
- Finish / colour / variant switching with price update
- Hotspots with product details, enquiry form to Supabase
- Screenshot / share view, analytics events (variant views, enquiries)
- White-label theming: logo, colours, domain-ready config file

Portfolio signal

Very few AI applicants also ship performant WebGL. A measured 60 FPS demo with a size budget and device testing proves a rare, hireable specialism - and it is already your studio's identity.

Build estimate: 10-14 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 4 - COPY-PASTE BUILDING PROMPT

3D Product Configurator / Virtual Showroom - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: 3D Product Configurator / Virtual Showroom - A 60 FPS WebGL configurator, white-
labelled for e-commerce and property - buyers change finishes and see the product, live.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js + TypeScript, Three.js / React Three Fiber, glTF models
(Blender, free), Draco compression, Supabase for product / lead data, Cloudflare Pages + R2 for
models. Free-tier end: R2 free storage / operations caps - compress models, lazy-load scenes,
set a per-scene size budget (target under 8 MB initial load).

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): brands, scenes,
products, variants, materials, hotspots, enquiries, analytics_events

API ROUTES (implement all, with input validation and structured errors): /api/products,
/api/variants, /api/enquiries, /api/analytics, /config/[brand] (theming)

MVP FEATURES (build ONLY these for v1): One hero product scene at 60 FPS on a mid-range phone
(show the FPS meter); Finish / colour / variant switching with price update; Hotspots with
product details, enquiry form to Supabase; Screenshot / share view, analytics events (variant
views, enquiries); White-label theming: logo, colours, domain-ready config file.

SPECIFIC RULES FOR THIS PROJECT: Performance is a feature, not a hope: add an on-screen FPS +
load-time readout in demo mode, a Lighthouse / WebGL budget check in CI, and fallbacks (static
poster image) for devices without WebGL2. Use only models you create in Blender or that carry a
clear commercial licence - log the licence per asset.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 5 OF 10

5 AI Lead Recovery & Outreach Agent

Revives dead CRM leads for agencies and real estate: enrich, score, draft a personal follow-up - human approves every send.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Agencies and property teams sit on hundreds of old enquiries they paid to acquire and never followed up past day two. Manual revival is tedious; fully automatic sending burns trust and domains.

Target customer

Lead-gen agencies, real-estate teams and coaching businesses with a spreadsheet or CRM full of stale leads.

> **Why it pays (targets, not guarantees):** Campaign setup target: $400-$1,200 per revival campaign (import, scoring, drafts, approval workflow). Retainer target: $199-$499/month for ongoing lists. Sell the approval gate as the safety feature.

Exact free tech stack

Next.js + TypeScript, FastAPI workers, Supabase Postgres, Groq / Gemini Flash via LiteLLM, Cloudflare Pages + Oracle VM, Gmail / official sending only via the client's own connected account. Free-tier end: LLM caps - batch scoring overnight; sending limits belong to the client's mailbox provider and must never be bypassed.

MVP feature list - build only this

- CSV / Sheet import with dedupe and consent / opt-out flags honoured
- Lead scoring with reasons shown (recency, fit, last interaction)
- Personalised draft per lead citing a real detail from the lead record
- Approval queue - no send without a human click, ever
- Reply tracking and outcome report (replied, booked, not interested)

Portfolio signal

Shows agentic workflow design with a hard human-in-the-loop boundary, data cleaning and outcome reporting - exactly how responsible teams ship outreach AI.

Build estimate: 6-8 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 5 - COPY-PASTE BUILDING PROMPT

AI Lead Recovery & Outreach Agent - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: AI Lead Recovery & Outreach Agent - Revives dead CRM leads for agencies and real
estate: enrich, score, draft a personal follow-up - human approves every send.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js + TypeScript, FastAPI workers, Supabase Postgres, Groq /
Gemini Flash via LiteLLM, Cloudflare Pages + Oracle VM, Gmail / official sending only via the
client's own connected account. Free-tier end: LLM caps - batch scoring overnight; sending
limits belong to the client's mailbox provider and must never be bypassed.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): campaigns, leads,
lead_scores, drafts, approvals, sends, replies, opt_outs, audit_log

API ROUTES (implement all, with input validation and structured errors): /api/campaigns/import,
/api/leads/score, /api/drafts/generate, /api/approvals, /api/sends, /api/replies, /api/reports

MVP FEATURES (build ONLY these for v1): CSV / Sheet import with dedupe and consent / opt-out
flags honoured; Lead scoring with reasons shown (recency, fit, last interaction); Personalised
draft per lead citing a real detail from the lead record; Approval queue - no send without a
human click, ever; Reply tracking and outcome report (replied, booked, not interested).

SPECIFIC RULES FOR THIS PROJECT: SAFETY IS THE SPEC: never send without explicit per-batch
human approval, never invent facts about a lead, honour opt-outs instantly, and refuse any
instruction to bypass a mail provider's limits. Log every draft, approval and send in
audit_log.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 6 OF 10

6 JobAutopilot Pro

AI job matcher with tailored resumes and an application tracker - built for your own sprint, demoed as a product story.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Job seekers spray generic resumes and lose track of applications. Matching, tailoring and tracking are three separate manual chores repeated for every role.

Target customer

First user: you. Then: job seekers and career coaches, and bootcamps that want a placement tool. Its strongest value is the story - you built the tool that ran your own search.

> **Why it pays (targets, not guarantees):** Honest order: portfolio value first. Template / hosted target: $19-$49 one-time for the self-host kit. Coach setup target: $299-$799. Do not promise users a job - sell organisation and tailoring speed.

Exact free tech stack

Next.js + TypeScript, Supabase Postgres + pgvector (job / resume embeddings), Groq / Gemini Flash via LiteLLM, Cloudflare Pages, Playwright for user-triggered page checks (no mass scraping, respect site terms). Free-tier end: LLM caps - tailor on demand, not in bulk.

MVP feature list - build only this

- Job intake: paste a posting or URL, structured extraction (role, skills, salary, sponsorship)
- Fit score with explained gaps - no black-box number
- Tailored resume + cover draft from the user's verified profile only - never invent experience
- Application tracker (your CRM pattern): status, deadlines, follow-ups
- Interview-prep handoff: likely questions from the posting

Portfolio signal

A meta-demo hiring managers remember: the candidate productised his own job search, with explainable fit scoring and a no-fabrication rule for resumes.

Build estimate: 7-9 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 6 - COPY-PASTE BUILDING PROMPT

JobAutopilot Pro - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: JobAutopilot Pro - AI job matcher with tailored resumes and an application tracker -
built for your own sprint, demoed as a product story.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js + TypeScript, Supabase Postgres + pgvector (job / resume
embeddings), Groq / Gemini Flash via LiteLLM, Cloudflare Pages, Playwright for user-triggered
page checks (no mass scraping, respect site terms). Free-tier end: LLM caps - tailor on demand,
not in bulk.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): users, profiles,
resumes, jobs, job_embeddings(vector), applications, tailored_docs, follow_ups

API ROUTES (implement all, with input validation and structured errors): /api/jobs/extract,
/api/jobs/score, /api/resumes/tailor, /api/applications, /api/follow-ups, /api/prep

MVP FEATURES (build ONLY these for v1): Job intake: paste a posting or URL, structured
extraction (role, skills, salary, sponsorship); Fit score with explained gaps - no black-box
number; Tailored resume + cover draft from the user's verified profile only - never invent
experience; Application tracker (your CRM pattern): status, deadlines, follow-ups; Interview-
prep handoff: likely questions from the posting.

SPECIFIC RULES FOR THIS PROJECT: HARD RULE: tailoring may re-order and re-word ONLY facts
present in the user's verified profile. If a posting requires a skill the profile lacks, list
it as a gap - never add it to the resume. No auto-apply: the user submits every application
themselves.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 7 OF 10

7 Faceless Shorts / Reels Factory (Autoshorts)

An automated short-video pipeline - script, voice, visuals, captions, render - for faceless YouTube and Instagram channels.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Creators need daily Shorts to grow, but scripting, voicing, editing and captioning one 45-second video takes an hour. Consistency, not talent, is the bottleneck.

Target customer

First: Signhify's own channels. Then: faceless-channel operators and agencies offering shorts packages to local brands.

> **Why it pays (targets, not guarantees):** Content-package service target: $199-$599/month per brand for a set number of shorts (you run the pipeline). Template target: $49-$129. Channel ad income is last and slowest - see the honest thresholds below.

Exact free tech stack

Python + FastAPI / workers, Kokoro TTS (open), faster-whisper locally for captions, FFmpeg rendering (NOT Remotion for commercial work - source-available licence), stock / generated visuals with licences logged, Oracle VM render queue, YouTube upload via the channel owner's own API credentials. Free-tier end: CPU rendering is slow - queue overnight, cap video length at 60 seconds.

MVP feature list - build only this

- Topic + script generator with hook in first 2 seconds, fact source required
- Kokoro voiceover + word-timed captions via Whisper alignment
- FFmpeg template engine: 3 visual templates, music from licence-clear sources only
- Approval queue before any upload - thumbnail, title, description
- Posting pack / upload via owner's own YouTube API; performance log

Portfolio signal

Shows media pipelines, queues and rendering at $0 - plus your Autogram / Autoshorts work becomes one coherent, documented system.

Build estimate: 6-9 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 7 - COPY-PASTE BUILDING PROMPT

Faceless Shorts / Reels Factory (Autoshorts) - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: Faceless Shorts / Reels Factory (Autoshorts) - An automated short-video pipeline -
script, voice, visuals, captions, render - for faceless YouTube and Instagram channels.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Python + FastAPI / workers, Kokoro TTS (open), faster-whisper locally
for captions, FFmpeg rendering (NOT Remotion for commercial work - source-available licence),
stock / generated visuals with licences logged, Oracle VM render queue, YouTube upload via the
channel owner's own API credentials. Free-tier end: CPU rendering is slow - queue overnight,
cap video length at 60 seconds.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): channels, topics,
scripts, voice_jobs, renders, approvals, uploads, performance_stats

API ROUTES (implement all, with input validation and structured errors): /api/topics,
/api/scripts/generate, /api/voice, /api/render, /api/approvals, /api/uploads, /api/stats

MVP FEATURES (build ONLY these for v1): Topic + script generator with hook in first 2 seconds,
fact source required; Kokoro voiceover + word-timed captions via Whisper alignment; FFmpeg
template engine: 3 visual templates, music from licence-clear sources only; Approval queue
before any upload - thumbnail, title, description; Posting pack / upload via owner's own
YouTube API; performance log.

SPECIFIC RULES FOR THIS PROJECT: YouTube Partner Program thresholds, stated honestly: 1,000
subscribers plus either 4,000 valid public watch hours in 12 months OR 10M valid Shorts views
in 90 days - and meeting them does not guarantee acceptance or income. Never use copyrighted
clips or music without a licence; log the source of every asset. Uploads only through the
channel owner's own credentials, after approval.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 8 OF 10

8 AI Meeting-to-CRM Copilot

Meeting notes in, structured CRM out: summary, decisions, action items with owners - ready to paste or push to the CRM.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

After every call, someone spends 20 minutes writing notes and still forgets action items. CRMs stay empty because updating them is manual.

Target customer

Agencies, consultants and small sales teams living in Google Meet / Zoom and a spreadsheet or lightweight CRM.

> **Why it pays (targets, not guarantees):** Setup target: $299-$899 per team (templates, CRM mapping, workflow). Template target: $39-$99. Care target: $49-$149/month for prompt / template tuning.

Exact free tech stack

Next.js + TypeScript, FastAPI, faster-whisper locally for uploaded recordings, Groq / Gemini Flash via LiteLLM, Supabase Postgres, Oracle VM. Free-tier end: Whisper on CPU is slow for long meetings - cap uploads, process in a queue, show progress.

MVP feature list - build only this

- Input: paste notes, upload transcript, or upload audio (transcribed locally)
- Structured output: summary, decisions, action items with owner + due date
- CRM-ready export: CSV / Sheet format + copy blocks per field
- Template per meeting type (sales, standup, client review)
- History search over past meetings (pgvector)

Portfolio signal

A clean, unglamorous productivity tool with structured extraction and evaluation potential - the kind of internal tool senior engineers are hired to build.

Build estimate: 5-7 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 8 - COPY-PASTE BUILDING PROMPT

AI Meeting-to-CRM Copilot - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: AI Meeting-to-CRM Copilot - Meeting notes in, structured CRM out: summary, decisions,
action items with owners - ready to paste or push to the CRM.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js + TypeScript, FastAPI, faster-whisper locally for uploaded
recordings, Groq / Gemini Flash via LiteLLM, Supabase Postgres, Oracle VM. Free-tier end:
Whisper on CPU is slow for long meetings - cap uploads, process in a queue, show progress.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): workspaces,
meetings, transcripts, summaries, action_items, exports, templates

API ROUTES (implement all, with input validation and structured errors): /api/meetings,
/api/meetings/[id]/transcribe, /api/meetings/[id]/summarise, /api/actions, /api/exports,
/api/search

MVP FEATURES (build ONLY these for v1): Input: paste notes, upload transcript, or upload audio
(transcribed locally); Structured output: summary, decisions, action items with owner + due
date; CRM-ready export: CSV / Sheet format + copy blocks per field; Template per meeting type
(sales, standup, client review); History search over past meetings (pgvector).

SPECIFIC RULES FOR THIS PROJECT: Never invent an owner or a due date: if the transcript does
not name one, output 'unassigned' / 'no date stated'. Keep the raw transcript beside every
summary so a human can verify in one click. Recordings are processed only with the uploader's
confirmation that participants consented.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 9 OF 10

9 Freelancer Client OS

Proposals, invoices, contracts and follow-ups in one micro-SaaS for freelancers and small agencies - your own studio workflow, productised.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Freelancers lose money in the gaps: slow proposals, forgotten follow-ups, late invoices, no single view of who owes what.

Target customer

Freelancers and 2-10 person agencies - starting with Signhify Studio itself, then your own network.

> **Why it pays (targets, not guarantees):** Self-host template target: $29-$79 one-time. Setup target: $149-$399 (branding, templates, import). Subscription only later, target $9-$19/month, after real users ask for hosted convenience.

Exact free tech stack

Next.js + TypeScript, Supabase Postgres + Auth, PDF generation with an open-source library (pdf-lib / WeasyPrint), Cloudflare Pages, Umami analytics. No LLM required for the MVP - add Groq drafting only as a v2. Free-tier end: Supabase caps - export / backup built in from day one.

MVP feature list - build only this

- Client + project pipeline board (your CRM pattern, simplified)
- Proposal builder from reusable blocks, PDF export
- Invoice generator with status tracking and due-date reminders list
- Follow-up sequences: draft reminders, human sends
- Dashboard: outstanding amount, proposals awaiting reply, overdue invoices

Portfolio signal

Shows you can build a complete, boring, correct business product - auth, PDFs, state machines, backups - the core of most paid full-stack work.

Build estimate: 5-7 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 9 - COPY-PASTE BUILDING PROMPT

Freelancer Client OS - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: Freelancer Client OS - Proposals, invoices, contracts and follow-ups in one micro-SaaS
for freelancers and small agencies - your own studio workflow, productised.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js + TypeScript, Supabase Postgres + Auth, PDF generation with
an open-source library (pdf-lib / WeasyPrint), Cloudflare Pages, Umami analytics. No LLM
required for the MVP - add Groq drafting only as a v2. Free-tier end: Supabase caps - export /
backup built in from day one.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): users, clients,
projects, proposal_blocks, proposals, invoices, invoice_items, follow_ups

API ROUTES (implement all, with input validation and structured errors): /api/clients,
/api/projects, /api/proposals, /api/proposals/[id]/pdf, /api/invoices, /api/invoices/[id]/pdf,
/api/follow-ups, /api/dashboard

MVP FEATURES (build ONLY these for v1): Client + project pipeline board (your CRM pattern,
simplified); Proposal builder from reusable blocks, PDF export; Invoice generator with status
tracking and due-date reminders list; Follow-up sequences: draft reminders, human sends;
Dashboard: outstanding amount, proposals awaiting reply, overdue invoices.

SPECIFIC RULES FOR THIS PROJECT: Money arithmetic must be exact and tested: unit tests for
totals, tax and discounts with two-decimal rounding. No payment processing in the MVP (no paid
gateway) - record payments manually. Every PDF must be generated from stored data, never from
client-supplied HTML.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

PROJECT 10 OF 10

10 AI Interview Prep Coach - Mobile-First PWA

Mock interviews with voice: answer out loud, get scored, specific feedback - a phone-first coach for job seekers.

$0 stack | Portfolio-ready | Copy-paste prompt inside

The problem

Candidates know the theory but freeze in spoken answers. Human mock interviews are expensive and hard to schedule; text-only prep does not train speaking.

Target customer

Job seekers preparing for AI / full-stack roles, bootcamp students, and career coaches who assign practice between sessions.

> **Why it pays (targets, not guarantees):** Portfolio + audience first. Premium-pack target: $9-$29 for question banks / advanced scoring (one-time, via Gumroad). Coach setup target: $199-$499. No outcome promises - never market it as 'get hired guaranteed'.

Exact free tech stack

Next.js PWA (installable, offline question bank) OR Expo / React Native - pick ONE; PWA is faster on free hosting. Web Speech API / faster-whisper for transcription, Kokoro for spoken questions, Groq / Gemini Flash for scoring, Supabase Postgres + Auth, Cloudflare Pages. Free-tier end: browser speech APIs vary by device - always offer typed-answer fallback.

MVP feature list - build only this

- Role tracks: GenAI Engineer, Full-Stack, plus custom posting paste-in
- Spoken or typed answers, timed sessions
- Scored feedback: structure, specificity, missing keywords - with the transcript shown
- Progress history and weak-topic list
- Offline question bank (PWA cache), no account needed for first session

Portfolio signal

Your mobile angle: a PWA with voice, scoring rubrics and offline support proves range beyond dashboards - and you can demo it on your own phone in an interview.

Build estimate: 7-10 days | **Prompt:** next page - copy it whole, do not edit the constraints out.

PROJECT 10 - COPY-PASTE BUILDING PROMPT

AI Interview Prep Coach - Mobile-First PWA - production prompt

Paste this entire block into your coding assistant (Claude / Cursor / Copilot) at the start of a fresh session in the project repo. It already contains the role, stack, architecture, schema, routes, tests, CI, free-tier deployment and the no-paid-services constraint.

```
ROLE: You are a senior full-stack + GenAI engineer building a production MVP for Signhify
Studio. Work in small, tested commits. Explain each step briefly, then write the code.

PROJECT: AI Interview Prep Coach - Mobile-First PWA - Mock interviews with voice: answer out
loud, get scored, specific feedback - a phone-first coach for job seekers.

HARD CONSTRAINT - NO PAID SERVICES: Use ONLY free and open-source technology. $0 budget. No
paid APIs, no paid tiers, no credit card. If a feature seems to need a paid service, replace it
with the free alternative named in the stack or cut the feature and say why.

STACK (use exactly this): Next.js PWA (installable, offline question bank) OR Expo / React
Native - pick ONE; PWA is faster on free hosting. Web Speech API / faster-whisper for
transcription, Kokoro for spoken questions, Groq / Gemini Flash for scoring, Supabase Postgres
+ Auth, Cloudflare Pages. Free-tier end: browser speech APIs vary by device - always offer
typed-answer fallback.

ARCHITECTURE: Monorepo with a Next.js + TypeScript web app (App Router), API routes as listed,
and a Python FastAPI service / worker only where the stack names one. Supabase Postgres with
Row Level Security on every tenant table. LLM access ONLY through a LiteLLM gateway module with
provider fallback (Groq free tier -> Gemini Flash free tier -> Ollama local) and a per-day
usage guardrail that stops calls before a free tier is exceeded. All secrets in environment
variables, never in code.

DATABASE SCHEMA (create as SQL migrations, with RLS policies and indexes): users, tracks,
question_banks, sessions, answers, scores, progress

API ROUTES (implement all, with input validation and structured errors): /api/tracks,
/api/questions, /api/sessions, /api/sessions/[id]/answer, /api/sessions/[id]/score,
/api/progress

MVP FEATURES (build ONLY these for v1): Role tracks: GenAI Engineer, Full-Stack, plus custom
posting paste-in; Spoken or typed answers, timed sessions; Scored feedback: structure,
specificity, missing keywords - with the transcript shown; Progress history and weak-topic
list; Offline question bank (PWA cache), no account needed for first session.

SPECIFIC RULES FOR THIS PROJECT: Scoring must be rubric-based and explainable: return the score
PER criterion with a quoted phrase from the user's answer as evidence. Never present the score
as a hiring prediction. Voice is optional at every step; the full flow must work typed-only.

TESTS + CI: Unit tests for core logic, API route tests, and one end-to-end happy path
(Playwright). GitHub Actions on every push: lint, typecheck, tests, production build.
Deployment must fail if tests fail.

DEPLOYMENT (free tiers only): Web on Cloudflare Pages; backend / workers on an Oracle Cloud
Always Free ARM VM with Docker Compose + Caddy; database on Supabase free (or self-hosted
Postgres + pgvector on the VM). Write the deploy steps into the README.

DELIVERABLES: Working deployed app, public GitHub repo, README with architecture diagram, setup
steps, free-tier limits table, demo script, and a 2-minute demo checklist. Ask me before adding
ANY feature outside the MVP list.
```

After it ships: deploy, record the 2-minute demo, add the free-tier limits table to the README, and log the project in your portfolio and CRM before starting the next build.

CLOSING PLAN

Your next 30 days: finish, prove, sell one

Days 1-11: Finish #1 - GTM Autopilot OS (Days 5-15)

No new project starts in this window. LLM adapters, approval UI, scheduler, publishers, analytics, hardening. Deploy, run it on Signhify's own content for a full week, record the demo. A finished flagship is worth more than all nine other ideas combined.

Days 12-21: Build #2 - RAG Support Copilot + Eval Dashboard

This is your interview weapon. Build the eval dashboard first-class, publish the groundedness numbers in the README, and write one short post explaining a failure case the evals caught. That post is outreach material for both jobs and clients.

Days 22-30: One services-led build - #3 or #5, chosen by a conversation

Do not choose by preference. Talk to three clinics / agencies first. Build the one a real buyer agrees to pilot. A pilot with a named business beats a polished demo with no user, every time.

> **Daily rule for the whole 30 days:** one build block, one distribution block. Distribution means a demo post, a portfolio update, an application or a client conversation - shipping silently does not compound.

How to present each project

- **GitHub README:** one-line pitch, live demo link, architecture diagram, stack + free-tier limits table, eval / performance numbers, setup in under 10 steps, licence.
- **Portfolio:** problem, your role, stack, one metric (FPS, groundedness %, hours saved in your own use), demo video under 2 minutes, link to repo.
- **LinkedIn:** one post per shipped project - the problem, a screenshot or clip, one honest number, what you would build next. No hype words, no income claims.
- **In applications:** name the project that matches the posting's stack, link the live demo first and the repo second, and state the constraint you built under ($0, free tiers) - it reads as resourcefulness.

GUARDRAILS

What NOT to build - and the final checklist

Skip these, however tempting

- **Paid-API wrappers:** a thin UI over one paid API fails your $0 rule and has no moat. If removing the API removes the product, there is no product.
- **Crypto / trading bots:** high risk, no credible portfolio signal for AI roles, and backtest screenshots convince no hiring manager.
- **Generic chatbots:** "Chat with your PDF" with no evals, no niche and no buyer is the most common portfolio project in the pile you are competing against.
- **Clones of big products:** a Notion / Twitter clone proves tutorials were followed. A small tool for a named buyer proves judgement.
- **Anything that needs a licence you do not have:** non-commercial model weights, fair-code tools resold as a service, scraped data against a site's terms. Check the licence before the first commit.

Ship checklist - every project, no exceptions

- Deployed on a free tier, with the tier's limits written in the README
- Public repo: tests passing, CI green, no secrets in history
- 2-minute demo recorded and linked on the portfolio page
- One honest metric published (eval score, FPS, load time, or your own time saved)
- Logged in your CRM / portfolio tracker with status and next action
- One distribution action done: post, application reference, or client conversation

> **The point of this document, in one line:** finish #1, prove #2, sell #3 or #5 - and let the job, the setup fees and the templates arrive in that order.

Prepared for Piyush Raj Singh, Signhify Studio - 7 November 2026. Prices and free-tier limits were stated as targets and current-at-writing figures; re-check provider limits and market pricing before quoting a client. No income is guaranteed by any project in this document.